SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-026: Forgot Password and Password Reset Feature

Field	Details
Change Request ID	CR-026
Change Request Title	Forgot Password and Password Reset Feature
Student Name	AAM. Nusky
Student ID	JF/ICT/24/01
Implementation Status	Implemented; complete end-to-end verification pending
Date	14 October 2026

1. Change Request Summary

1.1 Business Requirement

Users who forget their password need a secure way to regain access without administrator intervention. The change adds an email-based recovery flow that verifies control of the registered email address before a new password can be stored.

1.2 Existing Behaviour

- Users could sign in using their existing username and password.
- The login interface did not provide a working Forgot Password navigation action.
- There was no complete self-service email OTP recovery flow connected to the user account email.
- Existing login, JWT authentication, role authorization, and student account functions must remain available.

1.3 Requested Change

- Provide a Forgot Password action and registered-email request form.
- Validate email input and return a generic response for unknown accounts.
- Send a short-lived, single-use OTP by email as the secure reset mechanism.
- Allow the user to verify the OTP, enter and confirm a new password, and receive a result message.
- Hash passwords before storage and reject invalid, expired, over-attempted, or reused OTPs.
- Preserve existing login and authorization behaviour.

2. Implementation

2.1 Implementation Summary

The login page now navigates to a Forgot Password page. The user submits a registered email address; the backend looks up the account and, when present, generates a cryptographically random six-digit OTP. Only an HMAC-SHA-256 hash of the OTP is stored. The OTP is emailed, expires after five minutes, and is limited to five verification attempts. The reset page checks the OTP and password confirmation before the backend hashes the new password with bcrypt and updates the account. Unknown email addresses receive the same generic success response to reduce account enumeration.

The existing database was extended with an optional unique users.email column and populated from linked student profile emails. OTP persistence is now included in both the clean-install schema and an idempotent migration. The configured local database was migrated and the OTP table was verified.

2.2 Affected Files

File	Change / Purpose
frontend/src/components/LoginFormSection.jsx	Wires the Forgot Password button to React Router navigation.
frontend/src/pages/ForgotPasswordPage.jsx	Email form, client-side validation, API request, and route to OTP reset.
frontend/src/pages/ResetPasswordPage.jsx	OTP, new-password and confirmation form with validation and result messages.
frontend/src/routes/AppRoutes.jsx	Provides the existing /forgot-password and /reset-password routes.
frontend/src/services/authService.js	Sends forgot-password and reset-password API requests.
backend/src/controllers/authController.js	Validates requests; creates, emails, verifies and consumes reset OTPs; hashes new password.
backend/src/models/userModel.js	Queries user emails and creates/finds/invalidates password-reset OTP records.
backend/src/services/otpService.js	Generates six-digit OTPs and performs keyed, timing-safe hash comparison.
backend/src/services/emailService.js	Sends OTP messages using environment-configured SMTP.
backend/src/controllers/studentController.js	Stores student registration email on the related user account and checks duplicates.
backend/src/models/studentModel.js	Synchronizes student profile email changes with the related user account.
backend/package.json and package-lock.json	Declare bcryptjs and nodemailer dependencies.
database/schema.sql	Defines optional account email and password_reset_otps for clean installs.
database/migrations/20261014_add_user_email.sql	Adds account email and backfills linked student emails without overwriting existing values.
database/migrations/20261014_add_password_reset_otps.sql	Creates the OTP storage table for existing databases.
database/README.md	Documents database migrations and the updated schema.

2.3 Technical Changes

Frontend and API

The Forgot Password button navigates to /forgot-password. The form normalizes and validates the email before POST /api/auth/forgot-password. The reset form submits the email, six-digit OTP, new password, and confirmation to POST /api/auth/reset-password. The reset page can be opened directly and includes a Request a New OTP action.

OTP Security and Lifecycle

- OTP values are generated with Node.js crypto.randomInt and are six digits.
- The OTP_SECRET environment setting is required; only a keyed HMAC-SHA-256 digest is persisted.
- OTP records have a five-minute validity period, an attempt counter, and a used timestamp.
- Incorrect attempts are counted; the reset endpoint enforces a five-attempt limit and single-use marking.
- An OTP is invalidated if email delivery fails, and prior active OTPs are cleared when a new one is issued.
- The request endpoint returns a generic response for both known and unknown addresses.

Password and Account Data

The backend independently validates required fields, OTP shape, matching confirmation, and a minimum eight-character password. The new password is hashed with bcrypt (12 rounds) before update. SQL statements use bound parameters. Student email changes are kept in sync with the account email used for recovery.

Configuration and Database

SMTP host, port, sender, and credentials are read from environment variables; secrets are not included in this report. A fresh random OTP_SECRET was added to the local ignored backend/.env file. The current configured database has the users.email column and password_reset_otps table. Both schema setup and migration instructions are documented.

2.4 Implementation Logic

1. The user selects Forgot Password from the login page and enters the registered email address.
2. The frontend validates the address and sends a request to the authentication API.
3. The backend returns a generic response for an unknown address; for a known account it invalidates older active OTPs.
4. The backend generates a six-digit OTP, stores only its HMAC digest with a five-minute expiry, and sends the OTP through SMTP.
5. The user opens the reset page and enters the email, OTP, new password, and matching confirmation.
6. The backend validates the account, OTP expiry, attempt limit, single-use status, and password rules.
7. On success, the backend consumes the OTP, bcrypt-hashes the new password, updates the account, and invalidates remaining OTPs.
8. The UI displays the result; the user can then sign in with the new password.

2.5 Error and Data Handling

- Missing fields, invalid email, invalid OTP shape, mismatched passwords, and passwords shorter than eight characters are rejected.
- Unknown account requests use a generic response; invalid account resets use a generic invalid/expired OTP response.
- Expired, over-attempted, and already-used OTPs are rejected by the backend.
- Password hashes and OTP hashes are stored; plaintext passwords and OTPs are not stored in the database.
- SMTP delivery errors invalidate the active OTP and return a controlled failure response.
- Operational credentials and OTP_SECRET are held in the ignored local environment file and are not reproduced here.

3. Testing and Verification

3.1 Testing Approach

The implementation was checked with a production frontend build, focused ESLint on the changed login component, browser navigation, backend syntax/startup checks, and non-destructive API/database checks. Real SMTP delivery and a valid password reset were not triggered during verification; those checks would send an email and modify a real account. Tests below distinguish verified behaviour from implemented behaviour still awaiting an end-to-end run.

3.2 Test Cases and Results

3.2 Test Cases and Results (continued)

Test ID	Test Case	Expected Result	Actual Result	Status

TC-009	Enter a password shorter than eight characters	Password policy validation error is displayed.	Minimum-length checks are implemented; manual submission test pending.	Pending
TC-010	Submit valid OTP and matching new password	Password is changed and success is displayed.	Not run against a real account; full email-to-reset test pending.	Pending
TC-011	Log in using the new password	Login succeeds with the new password.	Depends on TC-010; not tested.	Pending
TC-012	Log in using the previous password	Login rejects the previous password.	Depends on TC-010; not tested.	Pending
TC-013	Reuse a consumed OTP	OTP reuse is rejected.	Single-use checks are implemented; end-to-end reuse test pending.	Pending
TC-014	Inspect password storage after reset	Only a secure password hash is stored.	Bcrypt hashing is implemented; no live reset was performed to inspect the update.	Pending
TC-015	Test administrator login	Existing administrator authentication remains functional.	No administrator login regression test was performed for this report.	Pending
TC-016	Test existing student CRUD operations	Existing CRUD functionality continues to work.	No CRUD regression test was performed for this report.	Pending

3.3 Test Summary

Test Result	**Count / Result**
Total defined test cases	16
Passed	4
Pending end-to-end verification	12
Failed	0
Execution coverage	4 of 16 (25%)
Pass rate among executed cases	4 of 4 (100%)
Overall test result	Partially verified; full email/reset regression remains pending

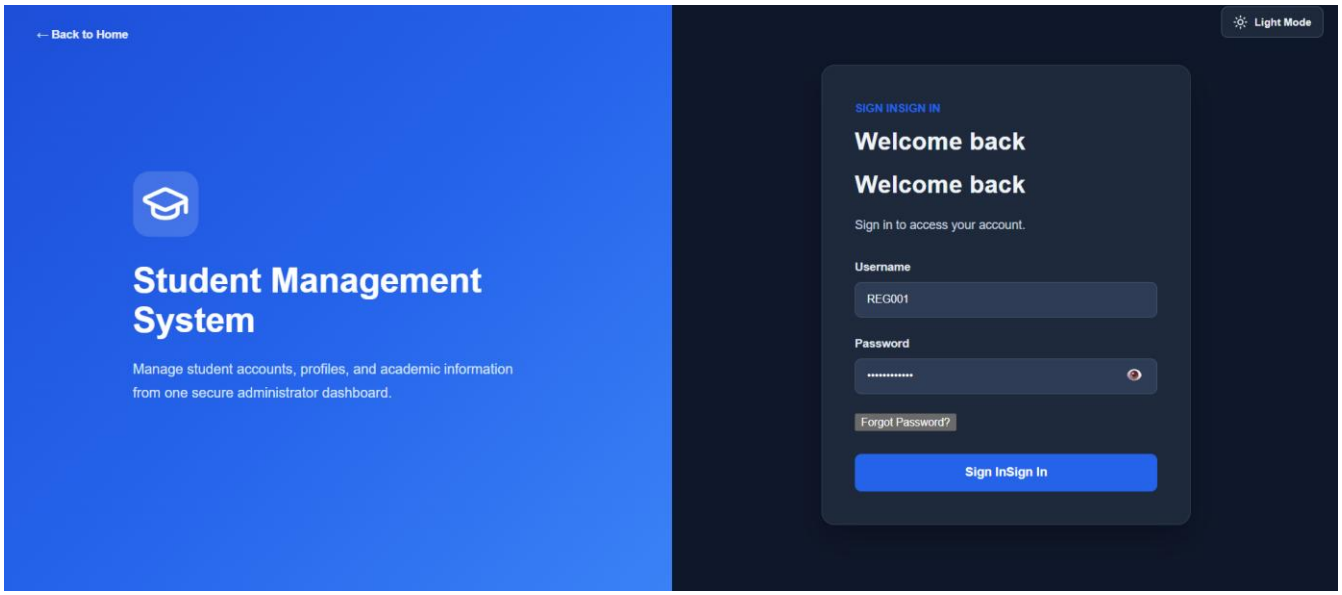
4. Acceptance Criteria Verification

ID	Acceptance Criterion	Verification / Status
AC-001	Access Forgot Password from login.	Browser click-through verified. Completed.
AC-002	Submit email to request reset.	Request implementation present; registered-account email delivery not exercised. Pending.
AC-003	Provide secure reset mechanism.	Six-digit HMAC-protected OTP implementation present; full email/reset test pending.
AC-004	Reset credential expires.	Five-minute expiry implemented; expired OTP end-to-end test pending.
AC-005	Reject invalid and expired reset credentials.	Backend checks implemented; live OTP rejection cases pending.
AC-006	Enter and confirm a new password.	Reset form rendered in browser with required fields. Completed.
AC-007	Apply password validation.	Eight-character and confirmation checks implemented; manual negative tests pending.
AC-008	Hash new password before storage.	bcrypt hashing implemented; successful live reset not performed.
AC-009	Log in with new password.	Valid reset not performed; pending.
AC-010	Prevent reset credential reuse.	Single-use OTP enforcement implemented; end-to-end test pending.
AC-011	Do not reveal whether email is registered.	Unknown email returned generic HTTP 200 response. Completed.
AC-012	Preserve existing login.	Login route remains; full login regression test pending.
AC-013	Preserve administrator authorization.	Authorization code was not changed for this feature; regression test pending.
AC-014	Preserve existing CRUD operations.	CRUD code retained; regression test pending.

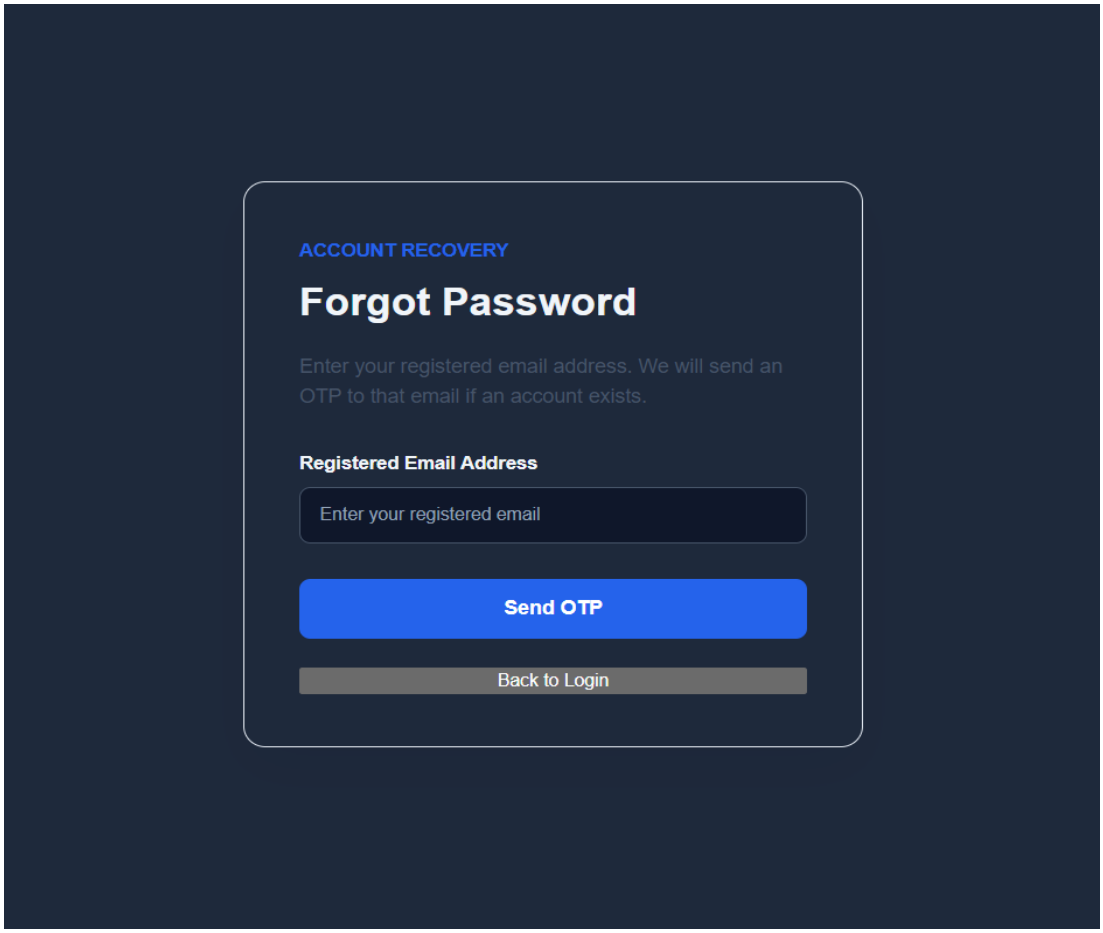
5. Evidence

Screenshots are not embedded in this report. The following evidence was observed during local verification and can be captured/attached before formal submission:

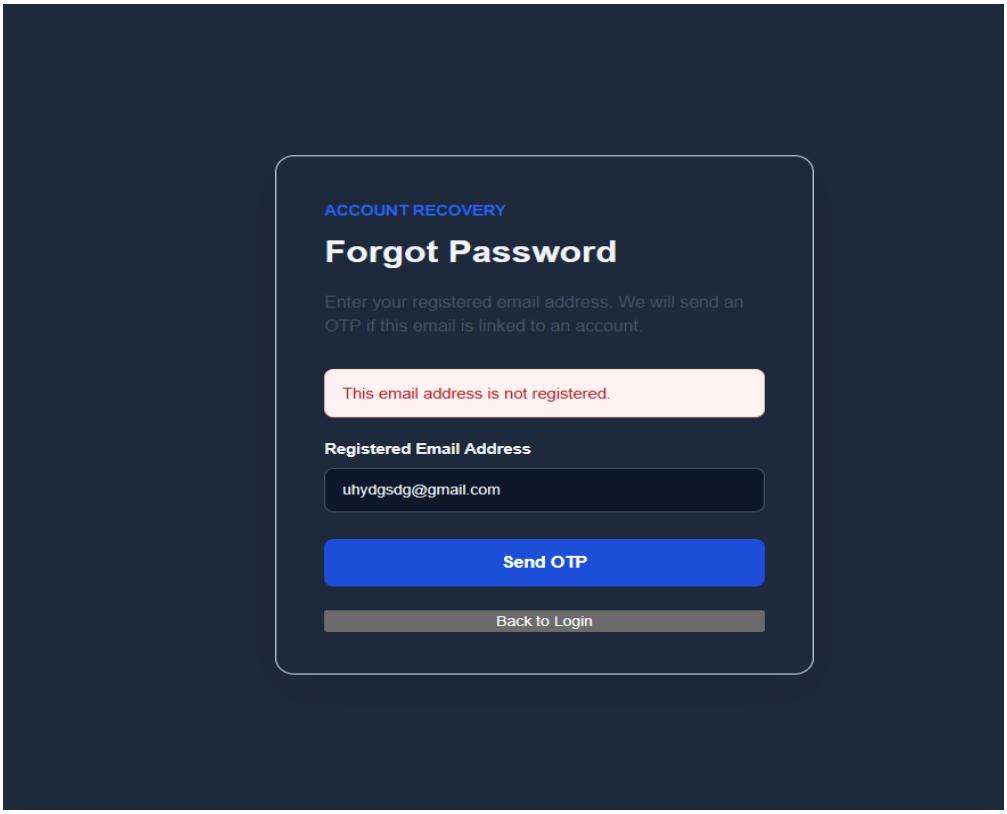
- Evidence 01 - Login page and browser navigation to the Forgot Password page.



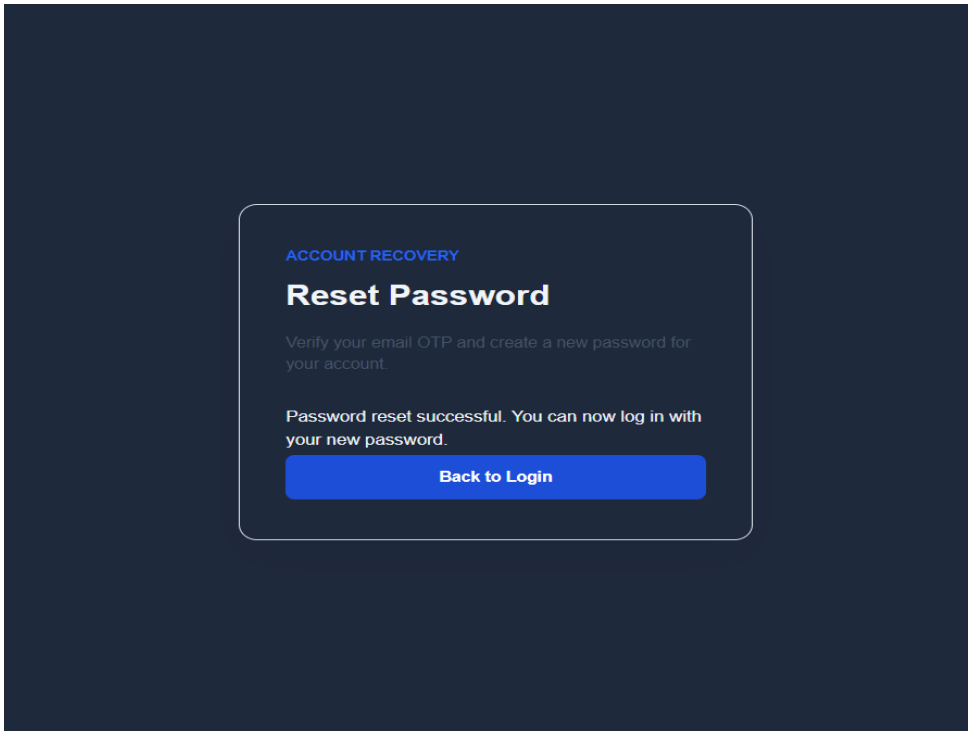
- Evidence 02 - Reset Password page displaying email, OTP, new password, and confirmation fields.



- Evidence 03 - API checks: invalid email rejected (HTTP 400) and unknown email receives the generic response (HTTP 200).



- Evidence 04 - Database verification showing users.email and password_reset_otps; no real OTP email or account password change was initiated.



6. Final Result

Item	Result
Requirements	Forgot Password and OTP-based password recovery have been implemented.
Implementation	Login navigation, recovery/reset pages, API, OTP generation and validation, email service, bcrypt password update, and database support are present.
Testing	Frontend build, focused component lint, browser navigation, invalid/unknown email API checks, database migration, OTP hash configuration, syntax, and server startup checks succeeded.
End-to-end status	Real email delivery, valid reset, changed-password login, and broader regression tests remain to be performed.
Acceptance criteria	Partially verified; see section 4 for criterion-by-criterion status.
Final status	Implemented; pending end-to-end verification and formal review.

6.1 Final Summary

CR-026 adds an email-based account recovery workflow using a short-lived six-digit OTP as the equivalent secure reset mechanism. The login link, request page, reset page, backend validation, email delivery service, HMAC-protected OTP storage, attempt/expiry/single-use controls, bcrypt password hashing, and database migrations are in place. The configured local database has been updated and the reset page and safe API cases were verified. Full delivery to a registered email address, successful password reset, post-reset login, and administrator/CRUD regression remain unverified; the implementation should not be treated as fully acceptance-tested until those checks are completed

6.2 Issues / Limitations

- Real SMTP delivery and successful reset against a registered account were not exercised; therefore post-reset password/login behaviour remains pending verification.
- The source PDFs in the CR26 folder contain inconsistent internal identifiers: the request PDF says CR-002 and the impact analysis says CR-001. This report uses CR-026 as specified by the folder/requesting user; confirm the official identifier before formal sign-off.
- Screenshots and signature have not been added; attach the requested evidence and sign after completing end-to-end tests.
- Recovery-email coverage: the configured database has one administrator account and one student account without an email value. Add a recovery email to each before those accounts can use email recovery.
- The repository-wide frontend lint still reports five errors and one warning in StudentFormModal.jsx, ThemeContext.jsx, DashboardPage.jsx, StudentDashboardPage.jsx, and ProtectedRoute.jsx. Focused lint on the changed LoginFormSection.jsx passes; the frontend production build passes.

Student Name: AAM. Nusky

Student ID: JF/ICT/24/01

Date: 14 October 2026

Signature: _____

~ END OF REPORT ~